

Kubernetes NGINX Ingress + Cloudflare + Cert-Manager SSL Setup SOP

1. Purpose

This SOP explains how to install NGINX Ingress Controller, configure Cloudflare DNS, install Cert-Manager, generate wildcard SSL certificates, and configure Kubernetes Ingress with HTTPS.

2. Prerequisites

- Kubernetes Cluster
 - Helm Installed
 - kubectl Configured
 - Domain registered (ayyarsachin.shop)
 - Cloudflare Account
 - Public IP Address / LoadBalancer IP
-

3. Install NGINX Ingress Controller

Add Helm Repository

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
```

Install Ingress Controller

```
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --create-namespace \
  --set controller.service.type=NodePort
```

Verify Installation

```
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
```

4. Configure Cloudflare DNS

1. Login to Cloudflare
2. Add domain: ayyarsachin.shop
3. Update nameservers at your domain registrar
4. Create DNS Records:

Type: A
Name: @
Value: YOUR_GCP_LB_IP

Type: A
Name: *
Value: YOUR_GCP_LB_IP

5. Create Cloudflare API Token

Cloudflare Dashboard → My Profile → API Tokens → Create Token

Use Template: **Edit zone DNS**

Permissions:

- Zone → DNS
 - Permission → Edit
 - Zone Resources → Include → Specific zone → ayyarsachin.shop
-

6. Install Cert-Manager

Install CRDs

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.crds.yaml
```

Add Helm Repository

```
helm repo add jetstack https://charts.jetstack.io  
helm repo update
```

Install Cert-Manager

```
helm install cert-manager jetstack/cert-manager \\\n  --namespace cert-manager \\\n  --create-namespace
```

7. Create Kubernetes Secret

```
kubectl create secret generic cloudflare-api-token \\\n  --from-literal=api-token=<YOUR_CLOUDFLARE_API_TOKEN> \\\n  -n cert-manager
```

8. Configure ClusterIssuer

Create file: clusterissuer.yaml

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-cloudflare

spec:
  acme:
    email: admin@ayyarsachin.shop
    server: https://acme-v02.api.letsencrypt.org/directory

    privateKeySecretRef:
      name: letsencrypt-cloudflare

    solvers:
    - dns01:
        cloudflare:
          apiTokenSecretRef:
            name: cloudflare-api-token
            key: api-token
```

Apply:

```
kubectl apply -f clusterissuer.yaml
```

9. Create Wildcard SSL Certificate

Create file: certificate.yaml

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: wildcard-ayyarsachin-shop
  namespace: default

spec:
  secretName: wildcard-ayyarsachin-shop-tls

  issuerRef:
    name: letsencrypt-cloudflare
    kind: ClusterIssuer

  dnsNames:
    - "*.ayyarsachin.shop"
    - "ayyarsachin.shop"
```

Apply:

```
kubectl apply -f certificate.yaml
```

10. Extract SSL Certificate and Key

```
kubectl get secret wildcard-ayyarsachin-shop-tls \
-o jsonpath="{.data.tls\.cert}" | base64 -d > cert.crt
```

```
kubectl get secret wildcard-ayyarsachin-shop-tls \
-o jsonpath="{.data.tls\.key}" | base64 -d > cert.key
```

11. Configure Kubernetes Ingress

Create file: ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```
metadata:
  name: app-ingress
```

```
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-cloudflare
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
```

```
spec:
  ingressClassName: nginx
```

```
  tls:
  - hosts:
    - app.ayyarsachin.shop
    secretName: wildcard-ayyarsachin-shop-tls
```

```
  rules:
  - host: app.ayyarsachin.shop
```

```
    http:
      paths:
      - path: /
        pathType: Prefix
```

```
      backend:
        service:
          name: demo
          port:
            number: 80
```

Apply:

```
kubectl apply -f ingress.yaml
```

12. Verification Commands

```
kubectl get ingress
kubectl get certificate
kubectl get pods -A
```

13. Expected Outcome

- NGINX Ingress Controller is running
 - Cloudflare DNS is configured
 - Wildcard SSL Certificate is generated
 - HTTPS enabled for <https://app.ayyarsachin.shop>
 - Traffic securely routed to Kubernetes services
-

14. Troubleshooting

```
kubectl logs -n cert-manager deploy/cert-manager
kubectl describe certificate wildcard-ayyarsachin-shop
kubectl describe clusterissuer letsencrypt-cloudflare
```

15. Security Recommendations

- Never expose Cloudflare API tokens publicly
 - Use Kubernetes Secrets for sensitive data
 - Rotate API tokens periodically
 - Enable HTTPS redirection
 - Use least-privilege permissions in Cloudflare
-